

Universidade Estadual de Londrina

Título do Trabalho

Marco Antônio Cottorello Henry

Londrina

1 de outubro de 2026

Sumário

1	Introdução	2
2	Relatório do mês de Abril	3
3	Relatório do mês de maio	7
4	Relatório do mês de junho	11
5	Relatório do mês de Agosto e Setembro	14

1 Introdução

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

2 Relatório do mês de Abril

A primeira atividade do mês de abril foi uma pesquisa sobre:

- **Malware** — programas maliciosos criados para danificar ou espionar computadores.
- **Ransomware** — tipo de vírus que “sequestra” os arquivos e pede dinheiro para devolvê-los.
- **Trojan (Cavalo de Troia)** — programa que se disfarça de algo útil, mas esconde uma ameaça.
- **Phishing** — golpe que tenta enganar a pessoa para roubar senhas ou dados (por e-mail ou site falso).
- **DDoS** — ataque que sobrecarrega um site com tantos acessos que ele sai do ar.
- **Man-in-the-Middle** — quando alguém intercepta a comunicação entre duas pessoas sem que elas percebam.

Cybersegurança: Vírus (Trojan)

Trojan é um tipo de malware que engana o usuário sobre sua verdadeira intenção. Geralmente, é espalhado por engenharia social, e-mails ou até anúncios falsos.

A origem do nome “trojan” ou “trojan horse” (em português, “cavalo de Troia”) vem da própria Guerra de Troia e do cavalo feito por Odisseu. Esse tipo de malware recebe esse nome porque se passa por algo legítimo, como processos de autenticação, com o objetivo de obter senhas dos usuários. Assim, com a senha e a conta, o invasor pode se passar pela vítima.

Hoje em dia, os métodos estão mais complexos: eles se disfarçam de programas legítimos. Contudo, em contraste com vírus e worms, não criam cópias de si mesmos — esse é o motivo de não serem considerados vírus propriamente ditos. Dessa maneira, instalam-se diretamente no computador de forma silenciosa.

A popularidade desse método se deve muito à sua facilidade de uso, pois qualquer pessoa pode acessar o computador de outra apenas com um arquivo. Eles também podem ter programações para realizar autodestruição após um comando ou determinado tempo.

As infecções podem ocorrer, normalmente, de três maneiras:

- **Engenharia social:** o usuário recebe um e-mail infectado ou sofre um ataque de phishing; ao clicar ou baixar o arquivo, acaba sendo infectado.
- **Downloads maliciosos:** geralmente por sites não confiáveis ou softwares piratas; ao baixar esses programas, a máquina passa a ser infectada.
- **Vulnerabilidades de segurança:** menos comuns, ocorrem quando o usuário não baixa nada, mas possui um sistema desatualizado ou antigo, tornando-se alvo de ataques.

Geralmente, após a instalação, o trojan possui três objetivos principais: roubar dados da vítima (como informações bancárias, senhas ou contas), controlar remotamente o dispositivo (podendo utilizá-lo em outros ataques) ou instalar outros malwares.

Na maioria dos casos, é difícil perceber se você foi vítima ou não. Contudo, há alguns sinais, como: baixo desempenho da máquina (travamentos recorrentes ou até a “tela azul”); alterações na área de trabalho e barra de tarefas (resolução, cor, ícones diferentes); presença de programas desconhecidos no gerenciador de tarefas; aumento de e-mails spam, anúncios e redirecionamentos para sites suspeitos.

Para evitar esses problemas, é recomendável: bloquear certos tipos de anexos em e-mails (como *.EXE, *.BAT, *.CMD, *.SCR e *.JS), evitar programas suspeitos, utilizar antivírus, usar VPNs e manter aplicativos e softwares sempre atualizados.

Alguns dos principais tipos:

- Backdoor: o atacante invade o computador infectado e pode fazer praticamente qualquer coisa, controlando-o remotamente. Pode enviar, receber, executar e apagar arquivos, além de reiniciar o sistema. É frequentemente usado para criar redes zumbi (botnets).
- Exploits: tiram proveito de vulnerabilidades em softwares ou aplicativos do computador infectado.
- Clampi: aguarda uma transação bancária para roubar dados da conta. É extremamente eficiente em se ocultar e burlar firewalls.
- Cryxos: a vítima recebe um aviso falso de infecção com um número de telefone; ao ligar, é chantageada por dinheiro ou induzida a conceder acesso à máquina.
- FakeAV: fingem ser antivírus. Assim como os cryxos, tentam extorquir dinheiro com ameaças falsas.
- IM: acessam logins de aplicativos de mensagens, como WhatsApp, podendo ser usados para extorsão, envio de malware ou até ataques DDoS.
- DDoS: realizam ataques distribuídos de negação de serviço, sobrecarregando um site com múltiplas solicitações, tornando-o indisponível.
- Mailfinder: roubam senhas de e-mail para enviar vírus aos contatos da vítima.
- Ransom: tornam o computador inutilizável e só restauram o acesso mediante pagamento, sendo usados em extorsões.
- Acesso remoto (RAT): permitem ao invasor controle total do dispositivo, possibilitando roubo de dados, espionagem e distribuição de outros malwares para formar botnets.

Referências:

- [eset]
- [wiki1]

- [kaspersky_trojan]

A segunda pesquisa do mês de abril foi: Entender o básico de como a internet funciona:

- Como dados viajam de um computador a outro de forma simplificada.
- O que é um endereço IP (o “endereço” do computador na rede).
- O que é uma porta (como “portas de entrada” para diferentes serviços).
- O que é cliente-servidor (um computador pede, o outro responde).

Migração de Dados

A viagem de dados ocorre por meio de redes físicas (cabos) ou sem fio (wi-fi), possuindo diversas maneiras de se transferir, convertendo informações de uma máquina para outra em sinais elétricos, luz ou rádio, sendo transmitidos e recebidos em pacotes. Esses pacotes podem seguir caminhos diferentes pela internet para evitar congestionamentos e, ao chegar ao destino, são organizados novamente na ordem correta. Isso pode ser feito de forma que se conecte a mais do que dois computadores, podendo conectar cada um em um certo computador, porém é uma forma muito complicada, por isso é utilizado um roteador que pode conectar a outros roteadores para transmitir esses dados de uma forma que necessite menos cabos, podendo se escalar a uma fonte infinita de computadores, sendo uma forma mais facilitada o uso de um provedor de serviços de internet (ISP) feito especificamente para isso, facilitando ainda mais a conexão.

IPs (Internet Protocol)

O endereço IP é usado para identificar um computador para a recepção ou o envio de dados de um dispositivo específico, possuindo geralmente um formato composto em série de 4 números separados por pontos, por exemplo: 192.123.5.29. Sendo uma forma eficiente para computadores, porém para humanos não possui essa vantagem, por isso IPs podem ser apelidados, conhecidos como nome de domínio, por exemplo google.com, facilitando a sua compreensão, assim como ajuda na sua localização visto que endereços IPs mudam toda hora. IPs também possuem diferenças quanto ao IP público e privado, sendo o público o endereço do roteador na internet, já o privado é o endereço da sua própria máquina. Os IPs também possuem tipos diferentes de formatos como o: IPv4 (Protocol Version 4): usa endereços de 32 bits (123.456.7.89); IPv6 (Protocol Version 6): usa endereços de 128 bits (alfanuméricos, 1234:db7:ff00:32:8764). Esses tipos possuem diferenças quanto a sua capacidade, formato, segurança (IPsec embutido em IPv6 e opcional em IPv4), configuração (IPv6 possui autoconfiguração automática) e cabeçalho (IPv6 mais simples).

Portas de Entrada (Portas de Protocolos de Rede)

Portas de redes são um meio para que um serviço funcione, direcionando aplicações e serviços para um destino receber dados de uma forma simplificada. Cada porta de entrada possui grupos diferentes, como por exemplo a porta 80/443 é usada para tráfego de web (navegador)

e a porta 22 usada para conexão segura (SSH). Cada porta possui 16 bits, existem 65534 portas possíveis que cada servidor pode usar, tendo outros protocolos usados por meio de uma autoridade de atribuição de números da internet (IANA), possuindo cada porta um tipo de serviço para que não haja conflitos ou imprevistos na internet, como: 53 = DNS; 67, 68 = DHCP; 80 = HTTP; 110 = POP3; 443 = HTTPS. O TCP e UDP são os protocolos mais básicos da comunicação de computadores. TCP: apresenta latência na transferência de dados, porém garante a entrega dos dados. UDP: transmite grandes quantidades de dados, porém não garante a entrega dos dados. Ambos os protocolos necessitam de endereço IP e portas para funcionar, e devido aos seus comportamentos é recomendado utilizar o IANA, tendo algumas portas que somente funcionam com TCP ou UDP, assim como TCP e UDP.

Cliente-Servidor

Cliente-servidor é um modelo de rede baseado em requisições e respostas: os clientes fazem as requisições de acesso aos serviços ou recursos e os servidores fornecem o serviço ou o recurso designado na rede, sendo necessária uma rede que permita a conexão entre os clientes e servidores.

Referências:

- Como a Internet funciona? — Aprendendo desenvolvimento web | MDN
- https://www.youtube.com/watch?v=W-_jjmkUH9k
- <https://www.youtube.com/watch?v=jWo792sVGBY&t=85s>
- <https://www.youtube.com/watch?v=FaTlrHq8RKM>
- <https://www.youtube.com/watch?v=Xmu-EpxcROM&t=147s>

A terceira atividade do mês de abril foi algumas revisões sobre Python.

Faça um programa que receba o raio r , e exiba o comprimento da circunferência C , a área do círculo A_c , a área superficial da esfera A_s e o volume da esfera V .

$$C = 2\pi r \quad A_c = \pi r^2 \quad V = \frac{4}{3}\pi r^3 \quad A_s = 4\pi r^2$$

```
raio = float(input('informe seu raio\n'))
pi = 3.1415
c = 2*pi*raio
v = (4/3)*pi*raio**3
ac = pi*raio**2
ass = 4*pi*raio**2
print(f"comprimeto: {c}\nvolume: {v}\narea do circulo: {ac}\narea
↪ superficial: {ass}")
```

Na primeira semana de maio, a atividade foi configurar o ambiente de desenvolvimento:

- Instalar o Python 3 (se necessário) e o VS Code com a extensão Python.
- Criar um ambiente virtual (venv): uma “caixa isolada” que guarda as bibliotecas do projeto sem afetar o resto do computador:
 - Criar: `python -m venv venv`
 - Ativar (Linux/Mac): `source venv/bin/activate`
 - Ativar (Windows): `.\venv\Scripts\activate`
- Entender o pip (gerenciador de pacotes) e criar um `requirements.txt`.

Nessa atividade, eu instalei uma venv no VS Code, com a versão 3.14.4 do Python. Aprendi também que para ativar o meu ambiente virtual eu utilizo o comando `.\venv\Scripts\activate`, e para desativar utiliza-se o comando `deactivate`.

3 Relatório do mês de maio

Neste mês fizemos os primeiros testes com a biblioteca `socket`, e já começamos a estruturar o chat com o modelo cliente e servidor.

Parte do cliente

```
import socket

HEADER = 64
PORT = 5050
FORMAT = 'utf-8'
DISCONNECT_MESSAGE = "!DISCONNECT"
SERVER = socket.gethostname(socket.gethostname())
ADDR = (SERVER, PORT)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(ADDR)

def send(msg):
    message = msg.encode(FORMAT)
    msg_length = len(message)
    send_length = str(msg_length).encode(FORMAT)
    send_length += b' ' * (HEADER - len(send_length))
    client.send(send_length)
    client.send(message)
    print(client.recv(2048).decode(FORMAT))

connected = True
```

```

while connected:
    msg = input()
    send(msg)

    if msg == DISCONNECT_MESSAGE:
        connected = False

client.close()

```

Parte do servidor

```

import socket
import threading

HEADER = 64
PORT = 5050
SERVER = socket.gethostbyname(socket.gethostname())
ADDR = (SERVER, PORT)
FORMAT = 'utf-8'
DISCONNECT_MESSAGE = "!DISCONNECT"

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(ADDR)

def handle_client(conn, addr):
    print(f"[NEW CONNECTION] {addr} connected.")

    connected = True
    while connected:
        msg_length = conn.recv(HEADER).decode(FORMAT)
        if msg_length:
            msg_length = int(msg_length)
            msg = conn.recv(msg_length).decode(FORMAT)
            if msg == DISCONNECT_MESSAGE:
                connected = False

            print(f"[{addr}] {msg}")
            conn.send("Message received".encode(FORMAT))

        conn.close()

def start():
    server.listen()
    print(f"[LISTENING] Server is listening on {SERVER}")

```

```

while True:
    conn, addr = server.accept()
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.start()
    print(f"[ACTIVE CONNECTIONS] {threading.active_count() - 1}")

print("[STARTING] server is starting...")
start()

```

Explicação do código

Este código implementa um servidor TCP concorrente utilizando as bibliotecas `socket` e `threading`. O objetivo do programa é permitir que vários clientes se conectem simultaneamente, criando uma thread para cada conexão.

Bibliotecas usadas:

- `import socket` — biblioteca responsável pela comunicação em rede. Permite criar servidores, criar clientes, enviar e receber mensagens.
- `import threading` — biblioteca responsável por criar *threads*, que permitem executar várias tarefas ao mesmo tempo. No projeto do chat: 1 cliente corresponde a 1 thread, e vários clientes a várias threads.

Configurações iniciais:

- `HEADER = 64` — define que os primeiros 64 bytes enviados conterão o tamanho da mensagem (ex.: mensagem “Olá” tem tamanho 3).
- `PORT = 5050` — porta utilizada pelo servidor.
- `SERVER = socket.gethostbyname(socket.gethostbyname())` — obtém automaticamente o endereço IP da máquina (ex.: 192.168.1.10).
- `ADDR = (SERVER, PORT)` — agrupa IP e porta (ex.: ("192.168.1.10", 5050)).
- `FORMAT = 'utf-8'` — define a codificação utilizada, convertendo texto em bytes.
- `DISCONNECT_MESSAGE = "!DISCONNECT"` — mensagem utilizada para encerrar a conexão.

Criação do servidor: `socket.socket(AF_INET, SOCK_STREAM)` cria um socket TCP, em que `AF_INET` indica IPv4 e `SOCK_STREAM` indica TCP. O TCP garante ordem, entrega e confiabilidade. Em seguida, `server.bind(ADDR)` associa o servidor ao endereço (IP + porta).

Função `handle_client()`: executada por cada thread, recebe `conn` (conexão com o cliente) e `addr` (endereço do cliente). Cada cliente conectado executa uma instância dessa função. O loop `while connected` mantém a comunicação ativa: recebe os primeiros 64 bytes que indicam o tamanho da mensagem, converte para número, recebe a mensagem real e a exibe.

Se a mensagem for **!DISCONNECT**, encerra a comunicação. A cada mensagem o servidor envia a confirmação “Message received” e, ao final, fecha a conexão com `conn.close()`.

Função `start()`: responsável por iniciar o servidor. `server.listen()` coloca o servidor em modo de espera e o loop `while True` o mantém funcionando continuamente. `server.accept()` espera um cliente conectar e retorna a conexão e o endereço.

Uso do `threading`: para cada conexão, `threading.Thread(target=handle_client, args=(conn, addr))` cria uma thread, em que `target` é a função executada e `args` são os argumentos enviados. `thread.start()` inicia a thread, atendendo o cliente em paralelo. `threading.active_count() - 1` conta quantas threads estão ativas, subtraindo 1 para ignorar a thread principal.

Fluxo geral:

```
Servidor inicia
-> listen()
-> Cliente conecta
-> accept()
-> Cria thread
-> start()
-> handle_client()
-> Recebe mensagens
-> Responde
-> Cliente desconecta
-> Fecha conexão
```

Este código implementa, portanto, um servidor concorrente usando Python, Socket e Threading.

A biblioteca `threading`

A biblioteca `threading` do Python funciona como um substituto à ideia original de projeto, que seria a criação de um servidor central. Essa biblioteca permite a execução de múltiplas tarefas simultaneamente dentro do mesmo programa por meio de threads (linhas de execução). Ela é muito utilizada em aplicações que precisam realizar várias operações ao mesmo tempo, como servidores, comunicação em rede, leitura de arquivos e espera por entrada do usuário.

No código, o `threading` foi usado para permitir que o servidor atendesse vários clientes ao mesmo tempo. Sempre que um cliente se conecta, o servidor cria uma nova thread responsável por tratar exclusivamente aquela conexão.

Principais funções utilizadas no código:

- `threading.Thread()` — cria uma nova thread. Exemplo: `thread = threading.Thread(target=handle_client, args=(conn, addr))`
- `target` — define qual função será executada pela thread.
- `args` — envia argumentos para a função executada pela thread.

- `start()` — inicia a execução da thread.
- `threading.active_count()` — retorna o número de threads ativas no programa.

4 Relatório do mês de junho

Nesse mês foram feitas algumas alterações no chat.

Nova versão do cliente

```
import socket
import threading
import time

nickname = input("Choose a nickname: ")

HEADER = 64
PORT = 5050
FORMAT = 'utf-8'
DISCONNECT_MESSAGE = "!DISCONNECT"
SERVER = "191.52.106.190"
ADDR = (SERVER, PORT)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(ADDR)

def receive():
    while True:
        try:
            message = client.recv(1024).decode(FORMAT)
            if message == 'NICK':
                client.send(nickname.encode(FORMAT))
            pass
        except:
            print("An error occurred")
            client.close()
            break

def send():
    connected = True

    while connected:
        time.sleep(0.8)
```

```

user_input = input("> ")

if user_input == DISCONNECT_MESSAGE:
    print("Disconnecting from the server...")
    connected = False
    msg = DISCONNECT_MESSAGE
    message = msg.encode(FORMAT)
    msg_length = str(len(message)).encode(FORMAT)
    msg_length += b' ' * (HEADER - len(msg_length))
    client.send(msg_length)
    client.send(message)
    time.sleep(1)

    client.close()
    break

else:
    msg = f"{nickname}: {user_input}"
    message = msg.encode(FORMAT)
    msg_length = str(len(message)).encode(FORMAT)
    msg_length += b' ' * (HEADER - len(msg_length))
    client.send(msg_length)
    client.send(message)

receive_thread = threading.Thread(target=receive)
receive_thread.start()

send_thread = threading.Thread(target=send)
send_thread.start()

```

Nova versão do servidor

```

import socket
import threading

HEADER = 64
PORT = 5050
SERVER = "0.0.0.0"
ADDR = (SERVER, PORT)
FORMAT = "utf-8"
DISCONNECT_MESSAGE = "!DISCONNECT"

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(ADDR)

```

```

server.listen()

clients = []
nicknames = []

def remove_client(client):
    index = clients.index(client)
    client.close()
    print(clients)
    nickname = nicknames[index]
    clients.remove(client)
    print(f"{nickname} desconectou do chat")
    broadcast(f"{nickname} left the chat".encode(FORMAT))
    nicknames.remove(nickname)

def broadcast(message):
    for client in clients:
        client.send(message)

def handle_client(client, addr):
    connected = True
    while connected:
        try:
            msg_length = client.recv(HEADER).decode(FORMAT)
            if msg_length:
                msg_length = int(msg_length)
                msg = client.recv(msg_length).decode(FORMAT)
                if msg == DISCONNECT_MESSAGE:
                    remove_client(client)
                    break
                broadcast(msg.encode(FORMAT))
        except:
            remove_client(client)
            break
    client.close()

def receive():
    while True:
        client, addr = server.accept()
        print(f"Connected with {str(addr)}")

        client.send("NICK".encode(FORMAT))
        nickname = client.recv(1024).decode(FORMAT)
        nicknames.append(nickname)

```

```

clients.append(client)

print(f"Nickname of client is {nickname}")
broadcast(f"{nickname} joined the chat".encode(FORMAT))
client.send("Connected to the server".encode(FORMAT))

thread = threading.Thread(target=handle_client, args=(client, addr))
thread.start()

print("[STARTING] server is listening...")
receive()

```

As principais atualizações feitas são:

- **Na parte do cliente:** Ao iniciar o código, o programa solicita que o usuário informe seu nome (nickname), que será utilizado para identificá-lo no chat. O cliente passou a enviar e receber mensagens simultaneamente por meio de duas threads: uma responsável pelo envio e outra pelo recebimento das mensagens. Além disso, ele passa a receber mensagens enviadas por qualquer cliente conectado ao servidor, permitindo uma comunicação em grupo.
- **Na parte do servidor:** O servidor passou a manter uma lista dos clientes conectados e seus respectivos nicknames, permitindo identificar o remetente de cada mensagem. Quando um novo cliente entra no chat, todos os participantes recebem uma mensagem informando sua conexão; da mesma forma, quando um cliente se desconecta, todos são notificados. Outra mudança importante é a utilização do endereço 0.0.0.0, permitindo que o servidor aceite conexões por qualquer interface de rede.

Novas funções:

- **receive():** executada em uma thread separada, fica constantemente aguardando e exibindo as mensagens recebidas do servidor, permitindo que o usuário continue digitando enquanto recebe novas mensagens.
- **broadcast():** envia uma mesma mensagem para todos os clientes conectados ao servidor, tornando possível o funcionamento do chat em grupo.
- **remove_client():** identifica o cliente que se desconectou, fecha sua conexão, remove-o das listas de clientes e nicknames e informa aos demais participantes que ele saiu do chat.

5 Relatório do mês de Agosto e Setembro

Nesses dois meses a pesquisa do projeto se concentrou na biblioteca pygames, para criar uma interface para o chat, e também para modularizar o código para melhorar sua organização e coesão.

As referências utilizadas para as pesquisas foram principalmente:

- <https://www.youtube.com/watch?v=AY9MnQ4x3zk&t=7762s>
- <https://medium.com/@habbema/modulariza%C3%A7%C3%A3o-em-python-9b4a8c250bfd>

O código já modularizado com a interface incluída ficou dividido em nove módulos que são ¹:

- client_handler
- manage_client
- client
- comunicacao
- config
- interface
- server_connection
- server_socket
- server

Nova versão do cliente modularizado

- client_handler

Este módulo é responsável por mandar as mensagens entre os clientes, codificando-as e decodificando-as ao chegar.

```
from config import HEADER, FORMAT, DISCONNECT_MESSAGE
from client_manager import broadcast, remove_client

def handle_client(client, addr):
    connected = True

    while connected:
        try:
            msg_length = client.recv(HEADER).decode(FORMAT)

            if msg_length:
                msg_length = int(msg_length)
                msg = client.recv(msg_length).decode(FORMAT)
```

¹todos os arquivos tem .py no final

```

    if msg == DISCONNECT_MESSAGE:
        remove_client(client)
        break

    broadcast(msg.encode(FORMAT))

except:
    remove_client(client)
    break

client.close()

```

- `manage_client`

Este módulo é responsável por gerenciar os usuários conectados, exibindo as mensagens de conexão e de desconexão. Ademais, também é aqui que fica a lista de clientes e de seus respectivos nomes.

```

clients = []
nicknames = []

def broadcast(message):
    for client in clients:
        client.send(message)

def remove_client(client):
    index = clients.index(client)

    client.close()

    print(clients)

    nickname = nicknames[index]

    clients.remove(client)

    print(f"{nickname} desconectou do chat")

    broadcast(f"{nickname} left the chat".encode("utf-8"))

    nicknames.remove(nickname)

```

- `client`

Está é a main do código do cliente, onde aqui são feitas as operações que unem os outros módulos, para que assim seja possível o funcionamento do chat, com o envio e recebimento de mensagens além da interface.

```
import pygame
import interface
import comunicacao

def main():
    interface.inicializar_interface()

    if not comunicacao.conectar():
        \texttt{print("Nao foi possivel conectar ao servidor.
↪ Encerrando...")
        interface.encerrar_interface()
        return

    nickname = ""
    texto_digitado = ""
    tela_atual = "nickname"

    rodando = True

    while rodando and comunicacao.conectado:
        for evento in pygame.event.get():
            if evento.type == pygame.QUIT:
                rodando = False

            elif evento.type == pygame.KEYDOWN:
                if tela_atual == "nickname":
                    if evento.key == pygame.K_RETURN:
                        if nickname.strip() != "":
                            tela_atual = "chat"
                            comunicacao.iniciar_receber(lambda: nickname) #labda cria
↪ funcoes sem nome(mais ideal pra coisas simples)
                    elif evento.key == pygame.K_BACKSPACE:
                        nickname = nickname[:-1]
                    elif evento.unicode.isprintable() and len(nickname) < 20:
                        nickname += evento.unicode

                elif tela_atual == "chat":
                    if evento.key == pygame.K_RETURN:
                        if texto_digitado.strip() != "":
                            msg = f"{nickname}: {texto_digitado}"
                            comunicacao.mandar(msg)
                            texto_digitado = ""
```

```

elif evento.key == pygame.K_BACKSPACE:
    texto_digitado = texto_digitado[:-1]
elif evento.key == pygame.K_ESCAPE:
    rodando = False
elif evento.unicode.isprintable():
    texto_digitado += evento.unicode

if tela_atual == "nickname":
    interface.desenhar_tela_nickname(nickname)
elif tela_atual == "chat":
    interface.desenhar_tela_chat(comunicacao.mensagens,
    ↪ texto_digitado)

interface.controlar_fps(60)

comunicacao.desconectar()
interface.encerrar_interface()

if __name__ == "__main__":
    main()

```

Nova versão do servidor

- server_connection

Este módulo é responsável pelo aviso de conexões do cliente com o servidor central, mostrando o nome de quem aderiu a conversa.

```

import threading

from config import FORMAT
from client_manager import clients, nicknames, broadcast
from client_handler import handle_client

def receive(server):
    while True:
        client, addr = server.accept()

        print(f"Connected with {str(addr)}")

        client.send("NICK".encode(FORMAT))

        nickname = client.recv(1024).decode(FORMAT)

```

```

nicknames.append(nickname)
clients.append(client)

print(f"Nickname of client is {nickname}")

broadcast(f"{nickname} joined the chat".encode(FORMAT))

client.send("Connected to the server".encode(FORMAT))

thread = threading.Thread(
    target=handle_client,
    args=(client, addr)
)

thread.start()

```

- **server_socket**

Módulo responsável pela integração da biblioteca socket, responsável pela possibilidade de intercomunicação, e criação do server em si.

```

import socket

from config import ADDR

def create_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server.bind(ADDR)

    server.listen()

    return server

```

- **server**

Main do server, que o inicializa e mostra a mensagem que ele está apto a receber clientes.

```

from server_socket import create_server
from server_connection import receive

server = create_server()

print("[STARTING] server is listening...")

receive(server)

```

Interface

- Explicação Toda a interface é baseada na biblioteca de python, Pygames. Sendo assim, o código consiste basicamente em duas telas, a primeira é uma onde o usuário informa seu nome e, após ele apertar Enter, é direcionado para a próxima. Essa nova tela é o chat em si, onde o usuário pode interagir com os outros.

A lógica por trás é simples: na tela inicial, após o usuário apertar Enter, ela é excluída e imediatamente repostada pela tela do chat. Além do mais, para fazer o efeito de subir das mensagens, é usada uma lista de palavras (as frases enviadas pelos usuários). Conforme vão sendo enviadas novas mensagens, seu eixo Y é atualizado, fazendo-as subir. Por fim, para não sobrecarregar a memória, quando seu Y passa o tamanho da tela, saindo da imagem, a frase é apagada da lista.

- Código

```

import pygame

AZUL = (1, 36, 86)
BRANCO = (245, 247, 250)

LARGURA, ALTURA = 800, 600
tela = None
fonte = None
relogio = None
altura_linha = 0
y_input = 0

def inicializar_interface():
    global tela, fonte, relogio, altura_linha, y_input
    pygame.init()
    tela = pygame.display.set_mode((LARGURA, ALTURA))
    pygame.display.set_caption("WhatsApp 2.0 :D")
    relogio = pygame.time.Clock()

```

```

fonte = pygame.font.SysFont("consolas", 24)
altura_linha = fonte.get_linesize()
y_input = ALTURA - altura_linha - 20

def desenhar_tela_nickname(nickname):
    tela.fill(AZUL)

    titulo = fonte.render("Whatsapp 2.0", True, BRANCO)
    tela.blit(titulo, (300, 150))

    texto = fonte.render("Digite seu nickname:", True, BRANCO)
    tela.blit(texto, (250, 250))

    nickname_surf = fonte.render(f"> {nickname}", True, BRANCO)
    tela.blit(nickname_surf, (250, 310))

    instrucao = fonte.render("Pressione ENTER para continuar",
↪ True, BRANCO)
    tela.blit(instrucao, (200, 400))

    pygame.display.flip()

def desenhar_tela_chat(lista_mensagens, texto_digitado):
    tela.fill(AZUL)

    y_atual = y_input - altura_linha
    for msg in reversed(lista_mensagens):
        if y_atual < 0:
            break
        msg_surf = fonte.render(msg, True, BRANCO)
        tela.blit(msg_surf, (10, y_atual))
        y_atual -= altura_linha

    texto_surf = fonte.render(f"> {texto_digitado}", True,
↪ BRANCO)
    tela.blit(texto_surf, (10, y_input))

    pygame.display.flip()

def controlar_fps(fps=60):
    relógio.tick(fps)

def encerrar_interface():
    pygame.quit()

```

Comunicação e Configurações

- Comunicação

Parte da lógica da comunicação entre chats, utiliza threads e sockets. A lógica da comunicação é similar as versões anteriores, porém agora modularizada.

```
import socket
import threading
import interface
from config import HEADER, PORT, SERVER, FORMAT,
↪ DISCONNECT_MESSAGE

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
conectado = False
mensagens = []

def conectar():
    global conectado
    try:
        client.connect((SERVER, PORT))
        conectado = True
        return True
    except Exception as e:
        print(f"Erro ao conectar com o servidor: {e}")
        conectado = False
        return False

def receber(obter_nickname):
    global conectado
    while conectado:
        try:
            message = client.recv(1024).decode(FORMAT)
            if message == 'NICK':
                nickname = obter_nickname()
                client.send(nickname.encode(FORMAT))
            else:
                mensagens.append(message)
        except:
            print("Conexao perdida com o servidor.")
            fechar_conexao()
            break
```

```

def iniciar_receber(obter_nickname):
    if conectado:
        thread = threading.Thread(target=receber, args=(
↪ obter_nickname,), daemon=True)
        thread.start()

def mandar(mensagem):
    if not conectado:
        return
    try:
        msg_encoded = mensagem.encode(FORMAT)
        msg_length = str(len(msg_encoded)).encode(FORMAT)
        msg_length += b' ' * (HEADER - len(msg_length))
        client.send(msg_length)
        client.send(msg_encoded)
    except Exception as e:
        print(f"Erro ao enviar mensagem: {e}")

def desconectar():
    if conectado:
        mandar(DISCONNECT_MESSAGE)
        fechar_conexao()

def fechar_conexao():
    global conectado
    conectado = False
    try:
        client.close()
    except:
        pass

```

• Configurações

O módulo de configurações serve para centralizar todas as informações vitais da rede do chat. Onde nele é definido o IP do servidor, o padrão para a codificação do texto, porta de comunicação, cabeçalho de envio de informações. A sua separação é importante para em caso de manutenções, troca de servidor por exemplo, fique mais fácil para serem feitas.

```

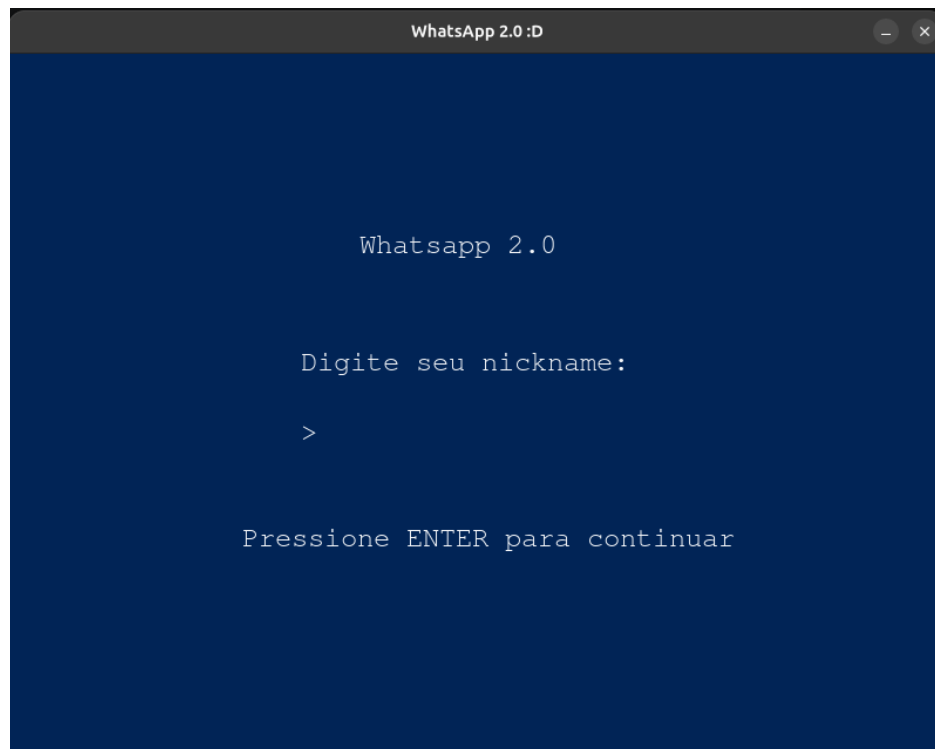
HEADER = 64
PORT = 5050
SERVER = "localhost"
ADDR = (SERVER, PORT)

```

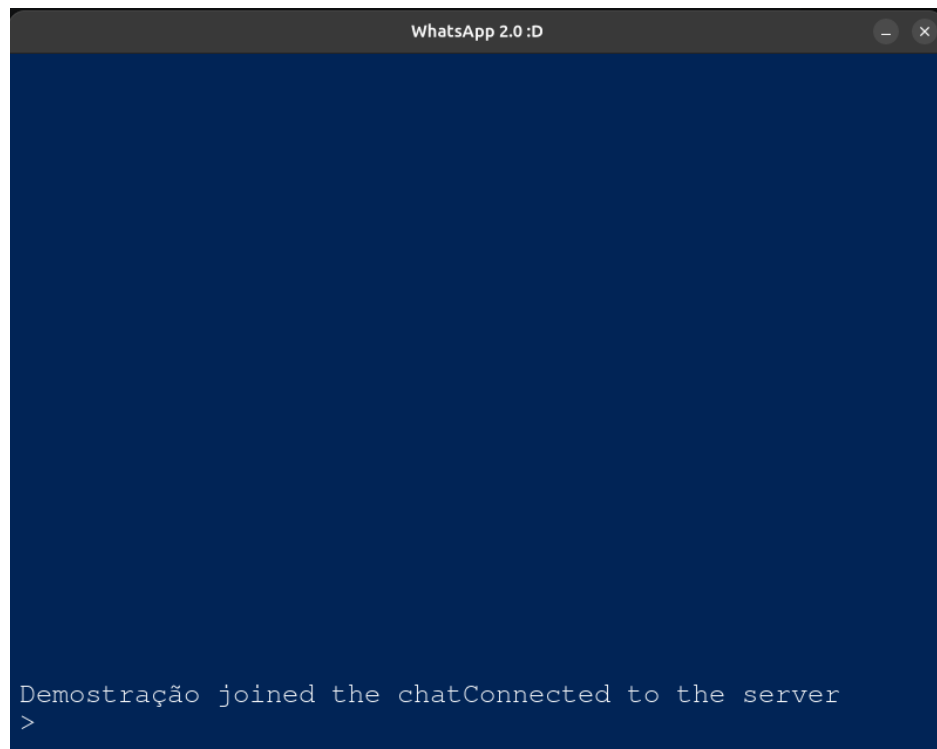
```
FORMAT = "utf-8"  
DISCONNECT_MESSAGE = "!DISCONNECT"
```

Demonstração de como ficou a interface

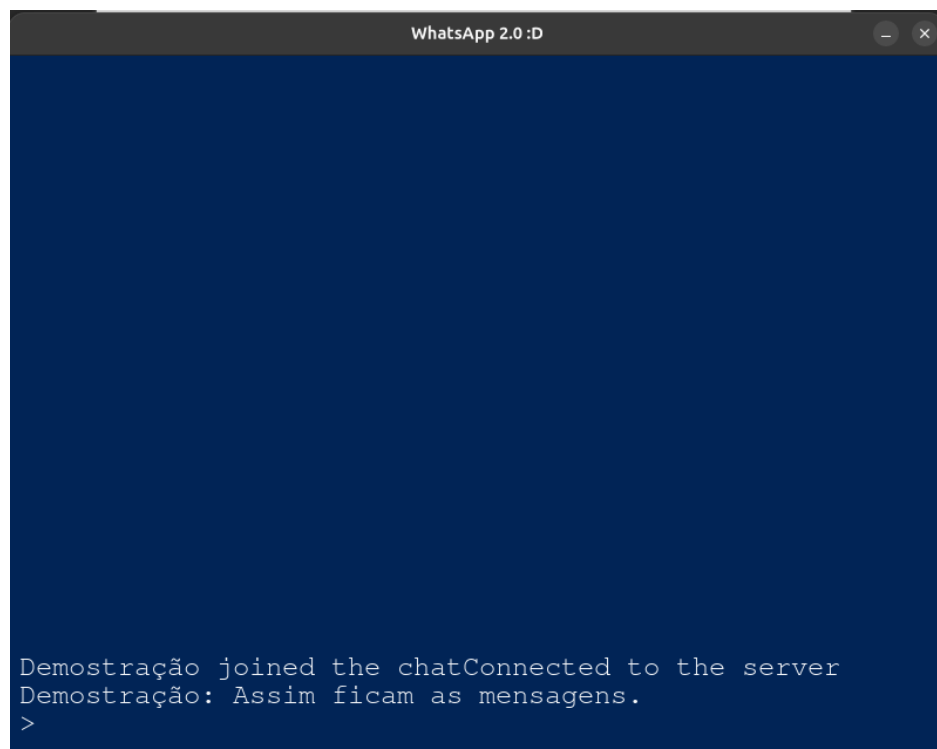
- Tela inicial do chat:



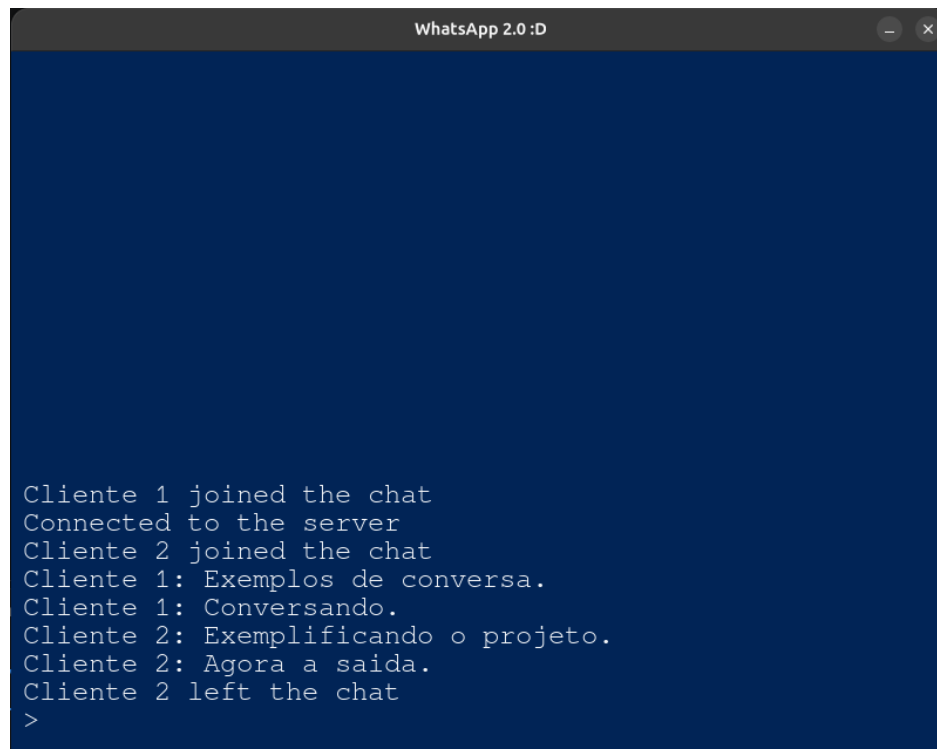
- Tela após entrar no chat:



- Exemplo de mensagem enviada:



- Exemplo de conversa e mensagem de saída do chat:



[kaspersky_trojan]

[mdn_internet]